

Travaux dirigés

Les algorithmes de tri

Complexité • Correction • Terminaison

CPGE — filières MP / PSI | Programme officiel marocain 2023

Conventions

- t (ou L) désigne un tableau (une liste Python) de n éléments deux à deux comparables ; on note $t[g..d]$ la tranche d'indices $g \leq k \leq d$.
- L'opération élémentaire de référence est la **comparaison** entre deux éléments ; on compte séparément les **échanges** et les **décalages** lorsque c'est pertinent.
- Une **inversion** de t est un couple (i, j) tel que $i < j$ et $t[i] > t[j]$. On note $I(t)$ leur nombre ; $0 \leq I(t) \leq \binom{n}{2}$.
- $H_n = \sum_{k=1}^n \frac{1}{k} = \ln n + \gamma + o(1)$; $\log$ est en base 2.
- Formule de Stirling : $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$; en particulier $n! \geq \left(\frac{n}{e}\right)^n$.

Méthode — les trois preuves attendues

Pour chaque algorithme on demande systématiquement :

Terminaison

un *variant* : quantité entière ≥ 0 strictement décroissante à chaque tour (boucle) ; ou une *fonction de mesure* à valeurs dans $\mathbb{N}$ strictement décroissante le long des appels (récursion).

Correction

un *invariant* de boucle : propriété vraie avant le premier tour (initialisation), conservée par un tour (hérédité), et qui donne le résultat voulu à la sortie (terminaison de la preuve) ; ou une récurrence bien fondée.

Complexité

le nombre d'opérations dans le meilleur cas, le pire cas, et lorsqu'on le demande en moyenne, avec la classe Θ finale.

Partie I — Les tris quadratiques

Exercice 1 — Tri par sélection : comptage exact et invariant

```

1 def tri_selection(t):
2     n = len(t)
3     for i in range(n - 1):
4         imin = i
5         for j in range(i + 1, n):
6             if t[j] < t[imin]:
7                 imin = j

```

```

8         t[i], t[imin] = t[imin], t[i]
9     return t

```

1. Donner le nombre *exact* de comparaisons $t[j] < t[imin]$ effectuées, en fonction de n . Dépend-il du contenu du tableau ?
2. Donner le nombre exact d'échanges. En déduire la complexité en Θ dans le meilleur et dans le pire des cas.
3. **Terminaison.** Justifier la terminaison des deux boucles.
4. **Correction.** Montrer l'invariant de la boucle externe :

avant l'itération i , la tranche $t[0..i-1]$ est triée par ordre croissant et contient les i plus petits éléments du tableau initial.

Conclure que l'algorithme trie t .

5. Pourquoi la boucle externe s'arrête-t-elle à $i = n - 2$ et non $i = n - 1$?

Exercice 2 — Tri par insertion : le rôle des inversions

```

1 def tri_insertion(t):
2     for i in range(1, len(t)):
3         cle = t[i]
4         j = i - 1
5         while j >= 0 and t[j] > cle:
6             t[j + 1] = t[j]
7             j -= 1
8         t[j + 1] = cle
9     return t

```

1. Dérouler l'algorithme sur $t = [5, 2, 4, 6, 1, 3]$: donner l'état du tableau après chaque itération de la boucle **for**.
2. **Meilleur cas.** Quel est le nombre de comparaisons si t est déjà trié ? En déduire la complexité.
3. **Pire cas.** Même question pour un tableau trié en ordre *décroissant* strict. Donner le nombre exact de comparaisons et de décalages.
4. **Cas général.** Montrer que le nombre total de décalages $t[j+1] = t[j]$ est exactement $I(t)$, le nombre d'inversions du tableau initial. *Indication* : montrer que chaque décalage supprime exactement une inversion, et qu'aucune opération n'en crée.
5. En déduire que la complexité du tri par insertion est $\Theta(n + I(t))$. Pourquoi dit-on que ce tri est « adapté aux tableaux presque triés » ? Que vaut la complexité si chaque élément est à distance au plus k de sa position finale ?

Exercice 3 — Tri par insertion : preuves complètes

On reprend le code de l'exercice 2.

1. **Terminaison de la boucle while.** Proposer un invariant et rédiger la preuve. Pourquoi l'accès $t[j]$ ne provoque-t-il jamais d'erreur d'indice ? (Le rôle de l'évaluation paresseuse du **and** en Python.)
2. **Correction de la boucle while.** Montrer l'invariant :

$t[j+2..i]$ contient, triés, les éléments de $t[j+1..i-1]$ initiaux, tous strictement supérieurs à cle ; et $t[0..j]$ est inchangé et trié.

3. **Correction de la boucle for.** Énoncer et démontrer l'invariant « $t[0..i-1]$ est trié et contient les i premiers éléments initiaux ». Conclure.
4. Justifier que le tri par insertion est un tri **en place**.

Exercice 4 — Tri à bulles et arrêt anticipé

```

1 def tri_bulles(t):
2     n = len(t)
3     for i in range(n - 1):
4         echange = False
5         for j in range(n - 1 - i):
6             if t[j] > t[j + 1]:
7                 t[j], t[j + 1] = t[j + 1], t[j]
8                 echange = True
9         if not echange:
10            return t
11    return t

```

1. **Correction.** Montrer l'invariant de la boucle interne : *après le tour d'indice j , $t[j+1]$ est le maximum de $t[0..j+1]$* . En déduire l'invariant de la boucle externe : *après l'itération i , la tranche $t[n-1-i..n-1]$ est triée et contient les $i+1$ plus grands éléments*.
2. **Terminaison.** Donner un variant pour chacune des deux boucles.
3. Donner le nombre de comparaisons dans le pire des cas (exact), et la complexité associée.
4. Grâce au drapeau `echange`, quelle est la complexité dans le meilleur des cas ? Donner une famille de tableaux l'atteignant.
5. Montrer que le nombre d'échanges effectués est exactement $I(t)$. Comparer avec le tri par insertion : lequel effectue le moins de comparaisons sur un tableau presque trié ?

Exercice 5 — Tri en place, tri stable

Définitions

Un tri est **en place** s'il n'utilise qu'un nombre $\Theta(1)$ d'emplacements mémoire auxiliaires, indépendant de n (la pile des appels récurifs comptant dans l'espace utilisé). Un tri est **stable** si, pour tout couple d'indices $i < j$ tel que $t[i]$ et $t[j]$ sont de clés *égales*, l'élément initialement en position i se retrouve, après le tri, à une position strictement inférieure à celle de l'élément initialement en position j .

1. Pourquoi la stabilité n'a-t-elle de sens que lorsque l'on trie des *enregistrements* selon une *clé* ? Donner une situation concrète où elle est indispensable (tri d'une liste d'élèves par note, puis par classe).
2. Montrer que le tri par sélection donné en exercice 1 n'est **pas stable** : exhiber un tableau de 3 éléments (clés notées $2_a, 2_b, 1$) qui le prouve.
3. Montrer que le tri par insertion **est stable**. Quel est le rôle exact du test $t[j] > cle$ (et non $\geq$) ?
4. Le tri à bulles est-il stable ? Justifier.
5. Compléter le tableau suivant (en place ? stable ?) pour les quatre tris rencontrés jusqu'ici, et le compléter à la fin du TD pour le tri fusion et le tri rapide.

Partie II — Le tri par fusion

Exercice 6 — La fonction `fusion` : spécification, invariant, variant

```

1 def fusion(A, B):
2     C = []
3     i, j = 0, 0
4     while i < len(A) and j < len(B):
5         if A[i] <= B[j]:
6             C.append(A[i]); i += 1
7         else:
8             C.append(B[j]); j += 1
9     return C + A[i:] + B[j:]

```

1. **Spécification.** Énoncer précisément : préconditions sur A et B , postcondition sur la valeur renvoyée.
2. **Terminaison.** Donner un variant de la boucle `while` et rédiger la preuve.
3. **Correction.** Démontrer l'invariant :

C est trié, C est formé exactement des éléments de $A[0..i-1]$ et $B[0..j-1]$, et tout élément de C est inférieur ou égal à tout élément restant de $A[i..]$ et de $B[j..]$.

En déduire la correction de `fusion`.

4. **Complexité.** On note $p = |A|$ et $q = |B|$. Montrer que le nombre de comparaisons c vérifie

$$\min(p, q) \leq c \leq p + q - 1.$$

Donner un exemple atteignant chacune des deux bornes. Conclure que `fusion` est en $\Theta(p + q)$.

5. Quelle est la complexité *spatiale* de `fusion` ?

Exercice 7 — `tri_fusion` : terminaison et correction

```

1 def tri_fusion(L):
2     if len(L) <= 1:
3         return L
4     m = len(L) // 2
5     gauche = tri_fusion(L[:m])
6     droite = tri_fusion(L[m:])
7     return fusion(gauche, droite)

```

1. **Terminaison.** Proposer une fonction de mesure et montrer qu'elle décroît strictement à chaque appel récursif. On vérifiera soigneusement que $0 < m < |L|$ dès que $|L| \geq 2$ — que se passerait-il si l'on avait écrit $m = 0$ ou $m = \text{len}(L)$?
2. **Correction.** Démontrer par récurrence forte sur $n = |L|$ que `tri_fusion(L)` renvoie une permutation triée de L .
3. `tri_fusion` est-il **en place** ? Évaluer l'espace mémoire total utilisé (tranches créées par $L[:m]$ et $L[m:]$, pile d'appels).
4. Dérouler l'arbre des appels sur $L = [5, 2, 4, 6, 1, 3]$: dessiner l'arbre de découpage puis les fusions successives.

Exercice 8 — Complexité du tri fusion

On note $C(n)$ le nombre de comparaisons dans le *pire* des cas pour n éléments.

1. Établir la récurrence $C(n) = C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + (n - 1)$ pour $n \geq 2$, avec $C(1) = 0$.
2. On suppose $n = 2^k$. Résoudre exactement la récurrence $C(2^k) = 2C(2^{k-1}) + 2^k - 1$ et montrer que

$$C(n) = n \log_2 n - n + 1.$$

Vérifier pour $n = 2$ et $n = 4$.

3. En déduire que le tri fusion est en $\Theta(n \log n)$ dans le pire des cas. Que vaut la complexité dans le meilleur des cas ? Le tri fusion est-il sensible à l'ordre initial des données ?
4. **Cas général.** Montrer par récurrence forte que $C(n) \leq n \lceil \log_2 n \rceil$ pour tout $n \geq 1$.
5. Comparer numériquement $n \log_2 n$ et $n^2/2$ pour $n = 10^3$ et $n = 10^6$.

Exercice 9 — Stabilité du tri fusion

1. Montrer que `tri_fusion` est stable. On identifiera précisément l'endroit du code qui assure cette propriété.
2. On remplace le test `A[i] <= B[j]` par `A[i] < B[j]`. L'algorithme reste-t-il correct ? Reste-t-il stable ? Donner un contre-exemple explicite avec des clés égales.
3. On remplace `m = len(L) // 2` par `m = 1`. L'algorithme reste-t-il correct ? Quelle est alors sa complexité ? Quel tri connu retrouve-t-on ?

Partie III — Le tri rapide**Exercice 10 — tri_rapide : preuves et cas extrêmes**

```

1 def tri_rapide(L):
2     if len(L) <= 1:
3         return L
4     pivot = L[0]
5     petits = [x for x in L[1:] if x <= pivot]
6     grands = [x for x in L[1:] if x > pivot]
7     return tri_rapide(petits) + [pivot] + tri_rapide(grands)

```

1. **Terminaison.** Proposer une fonction de mesure. Justifier soigneusement que $|\text{petits}| < |L|$ et $|\text{grands}| < |L|$ — où intervient le fait que le pivot est retiré de la liste ?
2. **Correction.** Démontrer par récurrence forte sur $|L|$ que `tri_rapide(L)` renvoie une permutation triée de L . On précisera pourquoi la concaténation `tri_rapide(petits) + [pivot] + tri_rapide(grands)` est bien triée.
3. **Meilleur cas.** On suppose qu'à chaque appel le pivot est un élément médian, c'est-à-dire $|\text{petits}| = \lfloor (n-1)/2 \rfloor$. Écrire la récurrence du nombre de comparaisons et la résoudre. Conclusion.
4. **Pire cas.** Montrer que si L est déjà trié par ordre croissant, alors `petits` est vide à chaque appel. Écrire la récurrence correspondante, la résoudre exactement et conclure. *Le tri rapide est donc quadratique sur une entrée déjà triée* : commenter ce paradoxe apparent.

5. Déterminer la profondeur maximale de la pile d'appels dans le meilleur et dans le pire des cas. Quel risque pratique cela représente-t-il pour $n = 10^6$?

Exercice 11 — Complexité en moyenne du tri rapide

On suppose que L est une permutation aléatoire uniforme de n éléments deux à deux distincts. On note $C(n)$ l'espérance du nombre de comparaisons, avec $C(0) = C(1) = 0$.

1. Justifier que le rang du pivot dans L suit la loi uniforme sur $\{0, 1, \dots, n-1\}$, puis établir

$$C(n) = (n-1) + \frac{1}{n} \sum_{k=0}^{n-1} (C(k) + C(n-1-k)) = (n-1) + \frac{2}{n} \sum_{k=0}^{n-1} C(k).$$

2. Multiplier par n , écrire la même relation au rang $n-1$ et soustraire pour obtenir

$$nC(n) = (n+1)C(n-1) + 2(n-1).$$

3. Poser $D(n) = \frac{C(n)}{n+1}$ et montrer que $D(n) = D(n-1) + \frac{2(n-1)}{n(n+1)}$. Décomposer $\frac{2(n-1)}{n(n+1)}$ en éléments simples.
4. En déduire $D(n) = 2H_n + \frac{4}{n+1} - 4$, puis

$$C(n) = 2(n+1)H_n - 4n$$

Vérifier la formule pour $n = 2$ et $n = 3$.

5. En déduire $C(n) \sim 2n \ln n = 2 \ln 2 \cdot n \log_2 n \approx 1,39 n \log_2 n$. Comparer au tri fusion ($\approx 1 \cdot n \log_2 n$) : de combien de pour cent le tri rapide effectue-t-il plus de comparaisons en moyenne ?
6. Le tri rapide est pourtant réputé plus rapide en pratique. Proposer deux raisons.

Exercice 12 — Partition en place (schéma de Lomuto)

La version par compréhension de l'exercice 10 n'est *pas* en place. On lui substitue la partition suivante, qui trie la tranche $t[g..d]$ en utilisant $t[d]$ comme pivot.

```

1 def partition(t, g, d):
2     p = t[d]
3     i = g - 1
4     for j in range(g, d):
5         if t[j] <= p:
6             i += 1
7             t[i], t[j] = t[j], t[i]
8     t[i+1], t[d] = t[d], t[i+1]
9     return i + 1
10
11 def tri_rapide_place(t, g, d):
12     if g < d:
13         m = partition(t, g, d)
14         tri_rapide_place(t, g, m - 1)
15         tri_rapide_place(t, m + 1, d)

```

1. Dérouler `partition` sur $t = [2, 8, 7, 1, 3, 5, 6, 4]$ avec $g = 0$, $d = 7$: donner l'état du tableau et les valeurs de i, j à chaque tour.

2. **Correction.** Démontrer l'invariant de la boucle **for** :

pour $g \leq k \leq i : t[k] \leq p$; pour $i + 1 \leq k \leq j - 1 : t[k] > p$; et $t[d] = p$.

En déduire qu'après l'échange final, l'indice $m = i + 1$ renvoyé vérifie : $t[m] = p$, tous les éléments de $t[g..m - 1]$ sont $\leq p$ et tous ceux de $t[m + 1..d]$ sont $> p$.

3. **Terminaison.** Justifier la terminaison de **partition**, puis celle de **tri_rapide_place** (fonction de mesure sur $d - g$).
4. Donner le nombre exact de comparaisons effectuées par un appel à **partition**(**t**,**g**,**d**).
5. Quelle est la complexité *spatiale* de **tri_rapide_place** dans le meilleur et le pire des cas ? Cet algorithme est-il en place au sens strict ?
6. Cette version est-elle stable ? Justifier par un contre-exemple.

Exercice 13 — Choix du pivot et entrées dégénérées

1. Soit t un tableau dont tous les éléments sont *égaux*. Montrer qu'avec la partition de Lomuto (test $t[j] \leq p$), la fonction **partition** renvoie toujours $m = d$. En déduire la complexité de **tri_rapide_place** sur un tel tableau. Même question pour la version par compréhension de l'exercice 10.
2. On choisit désormais le pivot *au hasard* parmi $t[g..d]$ (en l'échangeant d'abord avec $t[d]$). Le pire des cas $\Theta(n^2)$ existe-t-il encore ? Qu'est-ce qui change exactement ?
3. **Pivot médian de trois.** On prend pour pivot la médiane de $t[g]$, $t[(g + d)/2]$, $t[d]$. Montrer que sur un tableau déjà trié, la partition devient équilibrée. Existe-t-il encore des entrées quadratiques ?
4. **Partition ternaire (ouverture).** On partitionne en trois zones ($< p$), ($= p$), ($> p$) et l'on ne rappelle récursivement que sur les zones extrêmes. Quelle est alors la complexité sur un tableau constant ? Sur un tableau ne prenant que d valeurs distinctes ?

Partie IV — Borne inférieure, applications, synthèse

Exercice 14 — La borne $\Omega(n \log n)$: arbre de décision

On appelle *tri par comparaisons* tout algorithme dont les seules opérations sur les éléments sont des comparaisons deux à deux. Un tel algorithme, exécuté sur n éléments deux à deux distincts, se représente par un **arbre de décision** : chaque nœud interne est une comparaison $t[i] < t[j]$ (deux fils : oui / non), chaque feuille est une permutation renvoyée.

1. Dessiner l'arbre de décision du tri par insertion pour $n = 3$. Combien a-t-il de feuilles ? Quelle est sa hauteur ?
2. Justifier qu'un arbre de décision correct pour n éléments possède *au moins* $n!$ feuilles.
3. Montrer qu'un arbre binaire de hauteur h a au plus 2^h feuilles. En déduire $h \geq \log_2(n!)$.
4. À l'aide de $n! \geq (n/e)^n$, montrer que

$$h \geq n \log_2 n - n \log_2 e = n \log_2 n - 1,4427 n.$$

Conclure : tout tri par comparaisons effectue $\Omega(n \log n)$ comparaisons dans le pire des cas.

5. Le tri fusion est-il optimal au sens de cette borne ? Et le tri rapide ?
6. Pourquoi cette borne ne s'applique-t-elle pas à un tri qui utiliserait les éléments comme des *indices* de tableau (tri par comptage) ?

Exercice 15 — Trier pour résoudre

Dans tous les cas on comparera l'approche naïve et l'approche « on trie d'abord ».

1. **Doublons.** Écrire une fonction qui décide si un tableau de n entiers contient deux éléments égaux. Donner la complexité de la version naïve à deux boucles, puis celle de la version par tri. Conclure.
2. **Paire de somme donnée.** Étant donnés t et s , décider s'il existe $i \neq j$ avec $t[i] + t[j] = s$. Proposer un algorithme en $\Theta(n \log n)$ et justifier sa complexité (on pourra réutiliser la méthode des deux indices).
3. **Les k plus grands éléments.** Comparer : (i) k tours de tri par sélection, de complexité $\Theta(kn)$; (ii) un tri complet suivi d'une lecture, en $\Theta(n \log n)$. Pour quelles valeurs de k la première approche est-elle préférable ?
4. **Élément majoritaire.** Un élément est majoritaire s'il apparaît strictement plus de $n/2$ fois. Proposer un algorithme en $\Theta(n \log n)$ fondé sur le tri, et en prouver la correction.
5. **Médiane.** Quelle est la complexité du calcul de la médiane par tri ?

Exercice 16 — Synthèse et questions de cours

1. Compléter le tableau récapitulatif :

Tri	Meilleur	Moyenne	Pire	En place	Stable
Sélection					
Insertion					
Bulles					
Fusion					
Rapide (Lomuto)					

2. **Vrai ou faux**, en justifiant systématiquement :
 - (a) Un tri en place est nécessairement stable.
 - (b) Un algorithme en $\Theta(n \log n)$ est toujours plus rapide qu'un algorithme en $\Theta(n^2)$.
 - (c) Le tri rapide est en $\Theta(n \log n)$.
 - (d) Si $I(t) = O(n)$, le tri par insertion est linéaire.
 - (e) Aucun tri ne peut faire moins de $n \log_2 n$ comparaisons.
3. **Tri hybride.** On modifie `tri_fusion` : lorsque $|L| \leq S$ (seuil fixé), on trie L par insertion au lieu de découper.
 - (a) Justifier que l'algorithme reste correct.
 - (b) Montrer que la complexité dans le pire des cas est $\Theta(nS + n \log(n/S))$.
 - (c) Pour quel ordre de grandeur de S cette expression reste-t-elle $\Theta(n \log n)$? Que se passe-t-il si $S = \Theta(n)$?
 - (d) Pourquoi ce choix améliore-t-il les performances réelles alors que la classe de complexité est inchangée ?